

FRAME IT AS CASES

Work That Speaks for Itself · Week 2

Voice Card

Direct, curious, honest, technical, plain, no buzzwords.

Bio

I learn AI/ML by understanding problems and building working systems around them. My projects — a portable RAG artifact format and a two-stage hyperparameter search framework — are open source, and I document their limitations as carefully as their results.

Primary Action

[View my GitHub.](#)

RagBucket: rethinking what “portable” means for RAG

The Problem

When I build a traditional ML model, I can save it as a `.pkl` or `.pt` file and load it somewhere else. When I build a RAG system, I can't. After chunking documents, generating embeddings, and building a vector index, the whole retrieval setup stays tied to the project that built it. I kept asking myself: why can't I package the vectors, chunks, and retrieval config into one file and load it in another project? That question became RagBucket.

What I Did and Decided

I built a `.rag` artifact using Python's zipfile packaging — not a custom binary format. I wanted v1 to be simple and inspectable. The artifact holds a FAISS index (`vectors.faiss`), document chunks with metadata (`chunks.json`), and a manifest (`manifest.json`) describing the embedding and retrieval configuration.

I decided the artifact should own retrieval memory, but not the generation LLM. So the runtime can plug in Groq, OpenAI, Gemini, or Anthropic without rebuilding the artifact.

I originally built this around a fixed local embedding model (BAAI/bge-small-en-v1.5). While extending RagBucket to support multiple embedding providers, I realized the flaw: a query vector has to match the vector space the index was built with. Building with one embedding setup and querying with another is conceptually wrong. I didn't hit this as a production bug — I caught it while redesigning. I changed the artifact to store embedding config in the manifest, so the runtime always uses the artifact's own embedding setup at query time.

I also chose to keep v1 scoped to local FAISS only. I didn't try to support remote vector databases like Pinecone, because once retrieval state lives in external infrastructure, “package it into one file” stops being simple. That's still a real limitation.

What Came of It

RagBucket works end-to-end. I can build a `.rag` artifact from documents, load it with `RagRuntime` in a different project, and query it. It's published on PyPI and installable with `pip`.

I've proven the build-and-load workflow myself. I haven't proven portability across multiple machines or environments, and I don't have benchmark numbers yet — no artifact size, no load time, no Recall@K. I also don't know how many people have used RagBucket beyond installing it. That's a gap, not something I'm going to paper over.

Building this changed how I think about the word “portable.” I started out thinking portability meant putting the entire RAG system into one file. Now I think it means something narrower and more useful: making the pipeline reproducible and transferable — chunking strategy, embedding config, retrieval config, index compatibility — even in cases where the vector store itself can't live inside the artifact.

What I'd Do Differently

I'd design the artifact abstraction around that reframed definition of portability before building around FAISS, and I'd design evaluation into the project from the start instead of treating it as a follow-up.

LazyTune: a two-stage search pipeline that doesn't yet prove what I set out to prove

The Problem

Working with `GridSearchCV`, I noticed it fully trains every hyperparameter combination, even the ones that already look weak early on. With 4 values across 3 hyperparameters, that's 64 combinations, multiplied further by cross-validation folds. `RandomizedSearchCV` is faster, but it depends on which combinations get sampled — there's no ranking logic behind it. Bayesian optimization felt heavier than I wanted for a simple scikit-learn workflow. I kept thinking: why fully evaluate every configuration if some of them already look weak during an early screening stage? That question became LazyTune — screen and prune first, then only fully train the promising candidates.

What I Did and Decided

`SmartSearch` runs a two-stage process. First, every hyperparameter combination is evaluated with cross-validation (`cv_folds`, default 3) and ranked by mean CV score. Then a pruning step cuts the weak ones — either the `ratio` strategy (default `prune_ratio=0.5`, keep the top half) or the `adaptive` strategy (keep anything scoring at least 0.9 times the best screening score). Both are heuristics I chose for simplicity, not values I validated. Survivors get cloned, refit on the full training split, and evaluated on the held-out test set.

While explaining this project, I checked my own claim and found a flaw in it. I originally thought pruning before full training was the source of the speed advantage. But the screening stage already runs full cross-validation for every candidate — the expensive part isn't skipped, it's just not repeated a second time for the losers. Rough fit counts: LazyTune does $N \times cv_folds + survivors$ fits; `GridSearchCV` with `refit=True` does $N \times cv_folds + 1$. The current implementation does not give me evidence that it is faster than `GridSearchCV`, and structurally it may perform more fits depending on how many candidates survive pruning.

What Came of It

LazyTune runs end-to-end. You can pass a scikit-learn estimator and a parameter grid to `SmartSearch`, call `.fit()`, and get back the best params, best score, best estimator, and a summary. It's on GitHub.

I don't have a preserved benchmark — no fixed dataset, candidate count, survivor count, or held-out score I can point to. I've run it during development but didn't keep those runs as evidence. No one besides me has used it. And the harder gap: I don't have proof that the pruning heuristics (the `ratio` strategy at 0.5, or the `adaptive` strategy at a 0.9 threshold) actually keep the configurations that end up best on held-out data. They're implemented, not validated.

So the honest result: I built a working two-stage search pipeline — screening, ranking, pruning, full training, evaluation. I have not shown it's faster than `GridSearchCV`, and I have not shown the pruning heuristic reliably preserves the best candidates.

What I'd Do Differently

Make screening genuinely cheaper — using a data subset or a lower resource budget instead of full CV for every candidate — and validate the pruning heuristic before choosing default thresholds, instead of picking 0.5 and 0.9 and moving on. I would compare LazyTune against exhaustive search across multiple datasets and models, then measure whether the best configurations survive pruning, the final score difference, and the compute actually saved.

Before / After

Generic AI Line

“LazyTune is a cutting-edge hyperparameter optimization framework that dramatically speeds up model tuning through intelligent pruning.”

My Edited Version

“LazyTune screens every hyperparameter combination with cross-validation, then prunes the weak ones before full training. I don't have evidence it's faster than GridSearchCV — the screening stage already does full CV for every candidate, so depending on how many survive pruning, it can end up doing more fits, not fewer.”

Why I Changed It

The first line sounds confident, but my current implementation and evidence do not support the speed claim. The edited version says what the pipeline actually does and keeps the limitation visible.